



如何基于 Pi 开发自己的 Agent 项目

02 组装 Pi: Tool、Context 与输出合约

大家好，我是居丽叶。

上一篇，我们建了一个 `pi-minimal` 工程，把 DeepSeek 接进 Pi，在终端看到了第一条流式回复。但问它“参加会议的差旅报销需要哪些材料”，它还只能提醒我们补充学校制度。这是因为代码里的 `tools: []` 留空，模型没有查询资料的入口，自然也给不出学校的正式规定。

本篇继续给这个 Agent 加上制度查询工具，让它先查资料、再组织回答，并检查回答里的引用。我们会遇到三个问题：

- 模型怎样调用 Tool；
- 查询结果怎样进入 Context；
- 最终的输出怎样交给后面的程序使用。

把这三件事都解决了，一个 Agent 才有了明确的业务接口。

下一篇，我们再把多个这样的 Agent 组织成 Multi-Agent 系统，让它们按各自的职责协作。

一、先给 Agent 一个能查资料的 Tool

我们沿用上一篇的 `pi-agent-core + pi-ai`，固定使用 `1.0.0`，Node.js 为 `22.19.0` 或更高版本。模型接入继续复用 `providers.ts`，本篇只增加工具、上下文处理和输出校验。

配套工程位于系列仓库的 `02_组装Pi/code/pi-tool-agent`，先进入目录安装依赖：

代码块

```
1 npm install --ignore-scripts
```

如果想自己从上一篇改起，需要额外在依赖里加入 `"typebox": "1.3.27"`。它用来声明参数和输出的数据结构。

先说明一下：本篇的两条差旅条款都是**虚构教学数据**，用来观察工具调用过程，不代表任何学校的正式规定。我们先用本地数组代替制度库，把 Loop 跑通，再考虑接数据库和检索服务。

1. 模型看到工具说明，程序执行工具函数

工具可以先拆成两部分看：一部分告诉模型“有什么能力、怎么传参数”，另一部分负责真正执行查询。

在 `tools.ts` 中声明参数：

代码块

```
1 import { Type } from "typebox";
2
3 export const retrieveSchema = Type.Object({
4   query: Type.String({ minLength: 1, maxLength: 200 }),
5   top_k: Type.Optional(Type.Integer({ minimum: 1, maximum: 3 })),
6 }, { additionalProperties: false });
```

这份 schema 规定：查询词必须是非空字符串，最多 200 个字符；返回条数可以不传，传了就必须是 1 到 3 的整数；额外字段不接受。它会作为工具声明的一部分提供给模型，Pi 也会在执行工具前校验模型生成的参数。

工具的装配方式如下，完整实现见配套工程：

代码块

```
1 const retrieveTool = {
2   // 工具声明：告诉模型有什么能力、怎么传参
3   name: "retrieve_documents",
4   label: "检索演示制度", // 供界面展示
5   description: "查询演示差旅制度，返回条款 ID、标题和原文；查不到时返回空列表。",
6   parameters: retrieveSchema,
7   // 实际执行查询
8   execute: async (_toolCallId, { query, top_k = 2 }, signal) => {
9     signal?.throwIfAborted();
10    const hits = documents
11      .filter(d => query.includes(d.tag))
12      .slice(0, top_k);
13
14    return {
```

```

15     content: [{
16         type: "text",
17         text: JSON.stringify({ documents: hits.map(({ tag, ...d }) => d) }),
18     }],
19     details: { count: hits.length },
20 };
21 },
22 };

```

`name` 是模型发起调用时使用的名称；`description` 说明这个工具适合什么时候用；`parameters` 约束输入；`execute()` 是收到合法参数后，由 Pi 调用的业务函数。`label` 主要供界面展示。

这里没有向模型发送查询函数的源码。模型读到的是工具声明，随后生成一个名称和参数，例如：

代码块

```

1  {
2    "name": "retrieve_documents",
3    "arguments": { "query": "会议差旅报销材料", "top_k": 2 }
4  }

```

Pi 根据名称找到工具，校验参数，再调用 `execute()`。完整代码还会保存本次返回条款的 ID 和原文，供后面的引用校验使用。换成真正的制度库时，可以把本地筛选替换为后端检索请求，模型仍然通过同一个工具接口工作。

不过，参数合法只说明格式过关。它无法证明用户有权访问某个部门的文件。用户身份和可访问范围应由已经完成鉴权的业务代码传入工具执行环境，后端继续检查；即使模型在参数里提出了某个部门 ID，也不能直接把它当成授权依据。

2. 工具结果要让模型读到依据

第一次接工具时，很容易在返回值的 `content` 里只写“查询成功，共找到两条”。

程序知道查询成功了，模型却没有读到条款。下一轮照样只能猜。

本例让 `content` 携带条款 ID、标题和原文：

代码块

```

1  {
2    "documents": [
3      {
4        "id": "demo-travel-01",
5        "title": "演示差旅制度",
6        "text": "差旅报销需提交审批单、交通票据和住宿票据。"
7      },

```

```

8      {
9          "id": "demo-travel-02",
10         "title": "演示差旅制度",
11         "text": "参加会议的差旅报销，还需提交会议通知。"
12     }
13 ]
14 }

```

注意返回值里的两个字段：`content` 是给模型看的结果，`details` 是给程序、日志或界面使用的附加数据。本例把数量放进 `details`，把条款放进 `content`。如果条款原文只放在 `details`，模型不会自动看到它。

把工具装进 Agent 的方式，与上一篇一样：

代码块

```

1  const agent = new Agent({
2      initialState: {
3          systemPrompt: "回答差旅问题前先检索，仅依据返回条款回答，并给出条款 ID。",
4          model,
5          tools: [retrieveTool],
6      },
7      streamFn: models.streamSimple.bind(models),
8      toolExecution: "sequential",
9  });

```

这里设置 `toolExecution: "sequential"`，让同一轮中的多个工具调用依次执行，便于观察调用顺序。

这次 `tools` 终于有东西了！不过，注册工具只提供调用入口，是否发起调用仍由模型决定。我们的示例用指令要求先查；如果业务规定“所有制度回答必须查库”，还要在程序里检查本次是否取得依据，或者直接由后端先检索，不能把这条规定只留在提示词里。

二、看一次 Tool Call 怎样走完 Loop

在运行脚本的同一个终端设置好上一篇使用的 `DEEPSEEK_API_KEY`，然后执行：

代码块

```

1  npm start -- "参加会议的差旅报销需要哪些材料？请先检索差旅制度。"

```

下面这张截图来自一次实际运行：模型在同一轮回复中发起了两次查询，因此出现了两个 `tool start`。两次执行都发生在第一个 `turn_end` 之前；随后下一轮模型回复生成答案。

```
PS D:\... \pi-agent-notes\02_组装Pi\code\pi-tool-agent> npm start -- "参加会议的差旅报销需要哪些材料？请先检索差旅制度。"

> start
> node index.ts 参加会议的差旅报销需要哪些材料？请先检索差旅制度。

[event] agent_start
[event] turn_start
I'll search the travel policy documents first.[tool start] retrieve_documents {"query":"参加会议的差旅报销需要哪些材料","top_k":3}
[tool end] retrieve_documents isError=false
[tool start] retrieve_documents {"query":"会议差旅报销材料 会议通知 发票","top_k":3}
[tool end] retrieve_documents isError=false
[event] turn_end
[event] turn_start
根据检索到的差旅制度条款，参加会议的差旅报销材料如下：

**基础材料**（依据 demo-travel-01）：
- 审批单
- 交通票据
- 住宿票据

**参加会议的附加材料**（依据 demo-travel-02）：
- 会议通知

即参加会议的差旅报销共需提交：审批单、交通票据、住宿票据，以及会议通知。

以上仅为演示制度（demo-travel-01、demo-travel-02）的原文内容；如贵单位有更细化的材料清单（例如会议签到表、会议费发票等），本次检索未返回相关条款，无法据现有资料确认。[event] turn_end
[event] agent_end
```

过程可以概括为：

本次运行：同轮两次查询，下一轮生成回答



一次模型回复可以包含多个 Tool Call，Pi 会按配置执行它们，再把结果一起带进下一轮模型请求。日志只展示了本次调用顺序，无法证明模型读完第一次结果后才决定发起第二次查询。

上一篇的一次 `prompt()` 主要是一轮模型回复。加上工具后，同一次 `prompt()` 可能包含多轮模型请求，每轮回复可能触发一个或多个工具调用。Pi 会处理这个循环，我们无须在 `execute()` 结束后再手动调用一次模型。

序篇提过 Agent、Turn、Message 和 Tool Execution，这里就能对上实际过程：**一整个运行是 Agent run；一轮模型回复及其触发的工具执行组成一个 Turn；Tool Call 位于 assistant 消息的内容块中；**

Tool Result 则是一条独立消息。

工具执行结束，仍然可能有下一轮请求。因此，`tool_execution_end` 只能证明这次工具调用结束，不能证明整个任务成功。配套脚本会等待 `agent.prompt()` 完成，再检查最后一条 assistant 消息有没有错误、是否被取消，以及是否有最终回答。

查不到资料时，Agent 怎么回答

找到条款，就依据原文回答并附上来源；查询正常完成却返回空列表，则说明现有资料不足。这里还要分清一种情况：查询接口报错，只能说明这次没有查成，无法据此判断制度里有没有相关规定。

工具应区分**空结果**与**执行失败**：空结果正常返回，接口报错则抛出错误或返回带 `isError: true` 的结果。回答也应分别说明“在当前可查询资料中未找到匹配条款”和“本次查询失败”，避免直接推断“学校没有相关规定”。

如果需要进一步确认学校是否确实没有相关规定，可以引入人工核实（Human-in-the-loop），由主管部门确认；得到明确答复后，再说明“目前尚无相关规定”。

三、Tool 返回之后，Context 多了什么

查出两条资料以后，下一轮模型看到的内容已经变了：除了原来的指令和用户问题，还增加了刚才的 assistant Tool Call 和工具返回的条款。

Pi 的 `agent.state.messages` 保存消息记录。1.0 有一个需要留意的变化：system 消息也在这份记录里，承载指令与工具声明。我们处理历史时，应把这些消息一起考虑。

1. 先看任务需要什么，再决定保留哪些历史

工具调用和对话会不断增加消息，但我们不必每次都把全部历史交给模型。怎样取舍，可以继续用差旅问题来判断。

如果用户第一次问“差旅报销需要什么材料”，当前问题和本次检索条款就能组成回答所需的输入。如果接着问“参加会议还要补什么”，我们还得保留前面的问答，让模型知道这句话是在追问差旅报销。如果聊了很久，用户早先确认的“出差类型是会议”仍然影响答案，就应把这个条件单独保存，避免它随旧对话一起被删掉。

可以按任务情况选择：

任务情况	上下文处理方式	需要注意什么
单次制度查询	当前问题+本次检索条款	没有旧历史需要裁剪，先控制条款长度
连续追问	保留近期完整问答和已确认条件	不能把“会议”等影响答案的条件丢掉
长对话		

	保存业务条件，必要时总结旧对话，保留近期原文	摘要可能遗漏信息，需要检查条件是否还在
一次任务反复调用工具	按 Token 预算保留完整调用记录，压缩较早的过程	工具返回可能很长，调用与结果要对应

这些方式可以组合。例如，把“出差类型=会议”保存为业务状态，保留最近几次问答，再为本次检索条款分配长度预算。**先判断哪些信息会影响下一步，再决定它们以原文、摘要还是业务字段的形式进入上下文。**

2. 在 Pi 里，怎样接入自己的裁剪策略

Pi 提供 `transformContext()`，允许我们在模型请求前处理消息。具体保留什么，需要应用自己决定。下面演示一种自定义写法：保留最近三个完整请求组，以及全部 system 消息。

“三组”是本篇自定义的教学参数，未经过效果评测，也未直接摘自官方 demo 或行政 Agent。默认脚本只提交一个问题，不会触发裁剪；配套测试会在同一个 Agent 上连续提交四个问题，验证最近三组被保留。

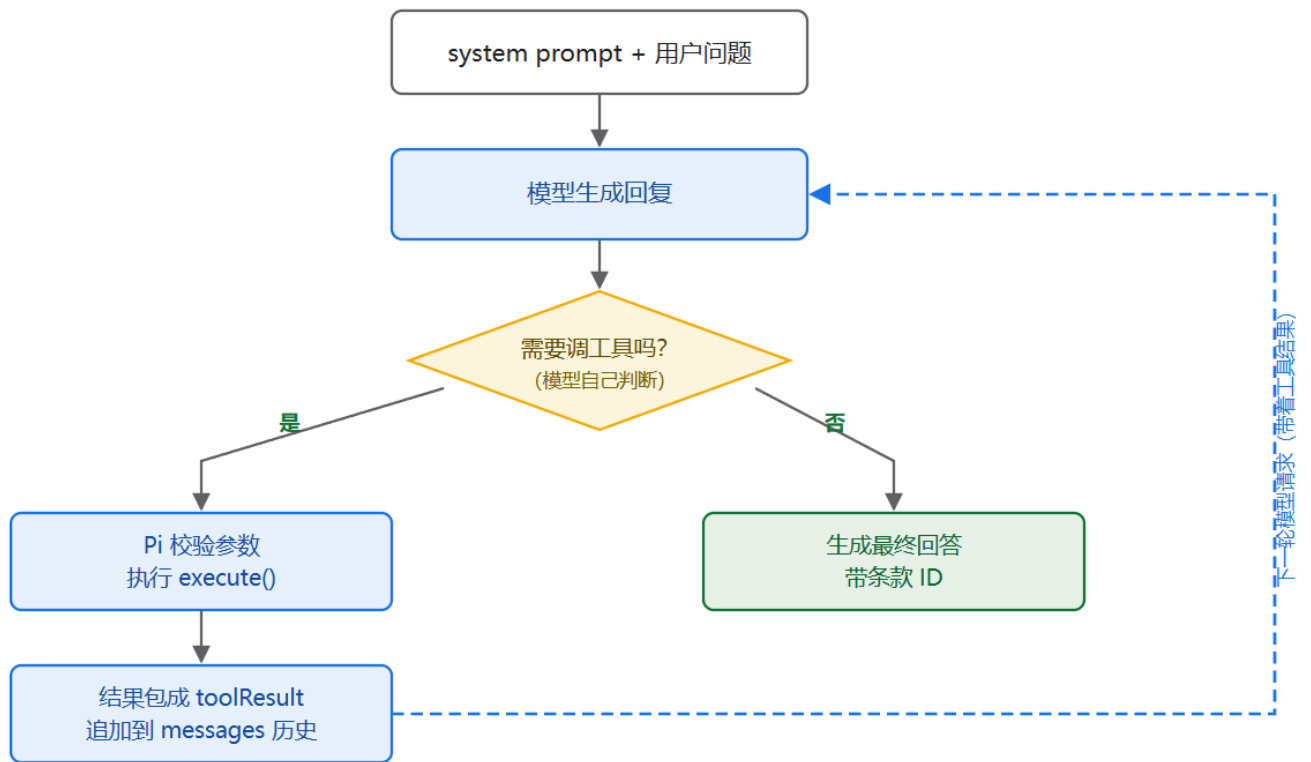
为什么按请求分组？因为消息记录里的“一条消息”不等于“一次完整问答”。例如：

代码块

```
1 user: 参加会议的差旅报销需要什么材料？
2 assistant: 调用 retrieve_documents
3 toolResult: 返回两条制度
4 assistant: 根据条款生成回答
```

直接用 `messages.slice(-10)`，只是取最后十条消息，切分位置可能落在一次工具调用中间，留下结果却丢掉调用记录。按 user 消息作为起点分组，则能在本例这种没有运行中追加输入的场景里，一起保留用户问题、模型回复和对应的工具结果。

一次 Tool Call Loop: 模型在循环里决定何时停



实线: 一次循环内部 蓝色虚线: 模型带着工具结果再次被调用 (可能多轮)
每多转一圈, messages 历史里就多一对 tool call / tool result

代码块

```
1 export async function keepRecentRequests(messages) {
2   const starts = messages.flatMap((m, i) => m.role === "user" ? [i] : []);
3   if (starts.length <= 3) return messages;
4   const cut = starts.at(-3);
5   return messages.filter((m, i) => m.role === "system" || i >= cut);
6 }
```

然后把它接进 Agent:

代码块

```
1 transformContext: keepRecentRequests
```

`transformContext()` 的结果再经过 `convertToLlm()`, 转成模型可接受的消息。这里返回筛选后的新数组, 没有直接删除 `agent.state.messages` 里的原始记录。保留全部 system 消息, 是为了让本例的指令与工具声明继续有效。

这个写法演示的是“按完整请求裁剪”, 不能保证 Token 不超限: 一个请求返回上百页原文, 即使只保留一组也可能装不下。接真实接口时, 需要限制工具返回的条数和长度, 并按模型窗口为指令、工

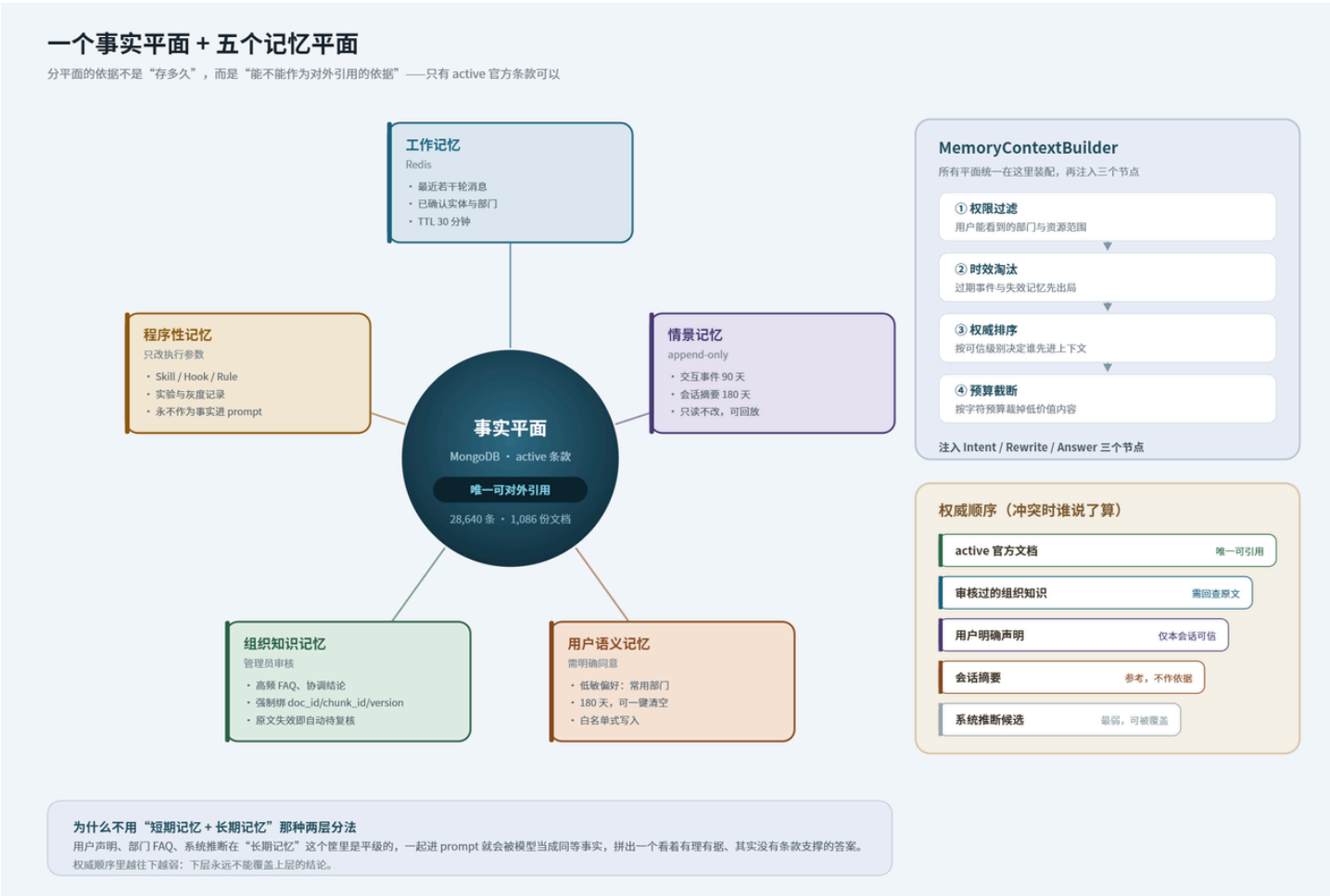
具声明、资料、历史和最终输出留出空间。加入运行中追加输入等能力后，还需要重新检查分组边界，确保每个工具调用都保留对应结果。

序篇介绍的自动压缩属于 Coding Agent 宿主的能力。我们现在直接使用 Agent Core，需要自己提供裁剪或摘要逻辑；创建一个 `new Agent()` 并不会自动获得完整的会话压缩系统。

3. 行政 Agent 为什么按业务任务重建上下文

我做的行政 Agent 主要处理“理解问题—检索制度—组织答案—检查依据”这类请求。Python 后端保存会话信息，再为每个角色组装本次需要的输入，因此无需让各个角色都接收整份 Pi 工具调用历史。

Context Builder 会组合会话摘要、已确认的业务条件、最近对话，以及召回的用户和组织记忆。比如用户追问“会议还要补什么”，近期对话帮助理解追问对象；如果出差类型已保存为业务条件，也可以直接补进输入；重新检索的条款则负责提供制度依据。



这张图展示的是设计概览：中间的“事实平面”提供可引用的制度原文，周围的记忆帮助理解用户，右侧的 Context Builder 选择各角色需要的信息。Answer 的输入明确规定，记忆不能作为制度事实或引用来源；重建上下文时还要检查会话归属和资料访问范围。

不过，当前实现仍有可以优化的地方：项目里的会话“摘要”是将最近四条消息各取前 240 个字符后拼接，尚未提炼和累计早期条件；记忆文本按字符预算删减，后续角色还会截取其中一部分。因此，长对话中的条件可能丢失。这份预算也只覆盖记忆部分，后续加入的问题、规则和制度条款仍需另行控制长度。

继续完善时，可以先做两件事：把影响答案的业务条件单独保存，并为最终发送给模型的整份输入分配预算。验收也可以看具体的追问：用户较早说过“参加会议”，聊了多轮后问“还要补什么材料”，答案是否仍考虑会议通知？这样才能判断上下文策略是否满足任务要求。

四、输出合约：让程序知道该接收什么

到这里，Agent 能查资料，也能根据资料回答了。给用户阅读时，可以返回带来源的文字；如果业务代码还要根据结果选择部门、调用下一个 Agent，或决定是否重试，就需要约定输出字段、字段含义和失败状态。

例如下一篇的 Router 要决定调用哪个 Worker，如果只输出“我觉得应该找财务处”，下游还得重新猜它的意思。更合适的是一个有类型约束的对象。

这就是本文所说的**输出合约**：调用方和 Agent 约定数据形状、状态含义、可接受的依据，以及不符合约定时的处理方式。这套业务接口需要我们在应用里设计，再用程序验收。

1. 给最终回答规定字段

接下来，我们让前面已经接好制度查询工具的 Agent，以 JSON 格式返回最终回答，并约定以下字段：

代码块

```
1  {
2    "status": "answered",
3    "answer": "请准备审批单、交通票据、住宿票据；参加会议还需会议通知。",
4    "citations": [
5      { "id": "demo-travel-01", "quote": "审批单、交通票据和住宿票据" },
6      { "id": "demo-travel-02", "quote": "还需提交会议通知" }
7    ]
8  }
```

查不到资料时，`status` 为 `insufficient_evidence`，`citations` 为空数组，`answer` 说明证据不足。这个对象是期望输出的示意，真实模型的措辞可能变化。

切换到 JSON 输出模式，在 PowerShell 中设置：

代码块

```
1  $env:OUTPUT_MODE = "json"
2  npm start
```

在 JSON 输出模式下，脚本会等待最终回答生成完毕，解析并校验通过后，再输出结果。

一次任务中，模型可能先输出“我查一下”，调用工具后再生成最终 JSON。如果把各轮输出的文字全部拼起来，就会得到“我查一下+JSON”，导致 JSON 解析失败。因此，配套脚本在运行结束后，只

提取最后一条 assistant 消息中的文本，用于解析和校验。

2. JSON 能解析，只过了第一关

我们的校验分三步：

1. 用 `JSON.parse()` 检查 JSON 语法；只兼容包住完整 JSON 的代码围栏。
2. 用 `TypeBox schema` 检查字段、类型和状态枚举；字段缺失或额外字段都拒绝。
3. 检查引用是否来自本次检索结果：工具返回条款时，程序保存每条的 ID 和原文；收到最终 JSON 后，检查引用 ID 是否存在，以及引文是否为对应原文中的连续片段。回答状态为 `answered` 时，必须有引用。

例如模型返回了 `demo-travel-99`，即使 JSON 格式完美，也会因本次检索没有这个 ID 而被拒绝。引用 ID 正确、引文却写成原文没有的“发票复印件”，同样被拒绝。

引用 ID 和引文都通过检查，也不代表结论正确。例如，条款要求提交会议通知，模型却回答“不需要提交会议通知”，即使附上真实引用，答案依然有误。下一篇会加入 Verifier，进一步检查答案是否与条款一致。

五、把示例接到真实业务

把本例的本地数组换成真实查询接口，就能沿用同一种装配方式，让 Agent 访问业务资料。我的行政 Agent 就采用了这条路线：`retrieve_documents` 调用 Python 后端的混合检索接口，再将返回的条款拼成 `[来源1] ..... [来源2] .....`，放进模型可读的 `content`。用户身份、资料权限和检索服务由后端管理，Pi 负责模型与工具之间的执行循环。

工具多起来后，也可以按角色分配。项目的 `services/pi-agent/src/agents.ts` 根据后端传入的 `allowedTools` 筛选工具，再创建 Agent。例如，识别问题涉及哪个部门的角色可以拿到 `list_departments`，需要补查制度或日期的角色可以拿到 `retrieve_documents` 或 `lookup_calendar`，无须给每个角色都开放全部工具。

接下来组织多个 Agent 时，还要分别约定它们的输入和输出。项目中的角色分工可以作为参考：

角色	本次需要的信息	输出给谁使用
Intent	用户问题、部门列表及必要的历史条件	输出 JSON，供业务代码选择后续路径
Rewriter	原始问题和术语表	输出 JSON，包含 1-3 个改写后的检索 query
Answer	当前问题、检索条款及必要的业务条件	输出带 [来源N] 引用的文本，供用户阅读
Verifier	用户问题、答案和对应条款	输出 JSON，供流程判断是否修正

本篇示例锁定 Pi 1.0.0，行政 Agent 仓库仍为 0.84.2，迁移时需按对应版本核对接口。项目已有 JSON 提取入口，本篇示例补充的字段与引用校验需另行接入。

六、先验收一个 Agent，再组织多个 Agent

跑通正常查询后，再用空结果模式确认它会说明证据不足；开启 JSON 输出，检查缺字段或不存在的引用能否被程序拒绝。换成自己的查询接口时，先替换 `execute()` 内部的实现，并保留这些验收条件。配套工程 README 中列出了运行方式和本地检查说明。

一个 Agent 的完成标准，应该是“执行了允许的操作，并交付了通过验收的结果”。终端里有一段流畅文字，只能说明模型生成了文字。

我的行政 Agent 项目把这些接口继续接到 Python 后端、部门检索、多角色协作和运行记录中。

如果你想看一套已经组装好的工程，可以用我做的行政 Agent，项目介绍见《[我做了一个自进化 Agent 项目，0 实习的同学也有救了](#)》。

写在简历上就是这样的：

项目经历

文枢 | 校内行政助手 Agent: 基于 Harness Engineering 的可信 RAG 与反馈自进化系统

2026.XX – 2026.XX

项目背景

接手学院与校网信息办已有的校内制度问答原型，原系统采用“文档切块 + 向量检索 + 单 Agent 生成”的简单 RAG 链路，存在多版本制度误引、纯向量检索精确信息不稳、Agent 执行不可控、用户反馈无法反哺策略等问题。针对教务、学工、财务、研究生院等多部门制度文档，参与升级为具备可信 RAG、Agent Harness、分层 Memory、反馈自进化 Loop 的校内行政助手 Agent。

解决方案

- **Agent Harness 编排治理**: 自建 Harness 层，将问答链路固定为 Intent → Rewrite → Retrieval → Answer → Verify DAG; pi Agent Runtime 负责意图识别、查询改写、答案生成、结果复核和 bad case 归因，自建治理层负责身份鉴权、部门隔离、事实校验、超时降级、灰度发布和自动回滚，避免 Agent 自由规划导致跳过检索或误答；
- **可信 RAG 数据链路**: 将 MongoDB 设计为事实平面，维护制度文档 active / archived 状态、版本链、来源哈希和条款元数据；Milvus 仅负责向量候选召回，命中后必须回 MongoDB 回填完整原始条款并校验生效状态，实现答案引用可核验；
- **混合检索与证据召回优化**: 针对制度问答中数字、电话、条款号、时间节点等精确信息难召回的问题，设计 BM25 + Milvus + RRF + Reranker 检索链路；BM25 负责关键词和精确信息召回，向量检索负责语义召回，RRF 融合后重排，并由 Query Rewriter 结合术语表和会话实体生成多组查询；
- **分层 Memory 架构**: 将记忆拆分为 Redis 工作记忆、情景记忆、用户语义记忆、组织知识记忆和程序性策略记忆；通过 MemoryContextBuilder 按权限、时效、权威级别和 token 预算裁剪上下文，避免会话摘要、部门 FAQ 或模型推断被误当成官方制度引用；
- **反馈自进化 Loop**: 实现 Execute → Observe → Reflect → Adapt → Deploy 闭环，采集点赞、点踩、纠错、追问、复制和 Verifier 结果；通过 Reflect 将 bad case 归因为 retrieval / intent / generation / knowledge_gap，并生成 Skill / Hook / Rule 候选，经历史 Trace 回放、稳定哈希灰度和指标监控后发布或回滚。
- **可执行 Skill Workflow**: 将 Skill 从普通提示词升级为可执行策略单元，支持查询词扩展、top-k 调整、回答模板切换、校历约束注入，并记录实验分组、触发次数、成功率和版本效果，用于验证反馈是否真实改变下一轮执行路径。

项目成果

- 在 1,086 份制度文档、28,640 条语义条款、1,200 条分层评测问题上完成规模化验收，Recall@5 达到 93.4%，MRR 0.862，引用正确率 96.1%，部门路由准确率 94.2%；
- 综合回答准确率从基础向量 RAG 的 82.6% 提升到 93.1%，其中混合检索与事实治理贡献 7.2 个百分点，两轮 Skill 灰度迭代贡献 3.3 个百分点。
- 无依据问题安全拒答率达到 97.3%，单部门问答 P95 3.6 秒，跨部门问答 P95 5.4 秒。
- 支持按部门拆分 Agent Service，通过 DEPT_ID 强制过滤和全局 Orchestrator 并行合并；完成 1 → 20 Pod 压测，200 并发下吞吐从 8.7 req/s 提升至 57.9 req/s。 (算法岗可以删除这一条)



本系列的配套代码也可以独立使用，先把自己的查询工具接上。

下一篇预告 | 03 组织 Pi: Router、Workers 与 Verifier

我们会在本篇的装配方式上创建多个 Agent: Router 分配任务，Workers 分别查询，Verifier 检查汇总结果。每个角色保留自己的 Context、Tool 和输出合约，调用顺序、并发与停止条件由外面的编排代码安排。